Greedy or Dynamic Programming? A Decision Framework

Programming and Data Structures — Week 10, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state a practical procedure for deciding whether greedy reasoning or dynamic programming is the appropriate technique for a new optimization problem; apply that procedure to a genuinely new problem — comparing fractional knapsack against Week 9's 0/1 knapsack — and explain the surprising result that the identical-sounding problem admits entirely different correct techniques depending on one small rule change; consolidate every proof technique covered across this unit into a single comparison; and explain why "try greedy first, fall back to dynamic programming if it fails" is a defensible practical strategy, along with the important caveat that makes it defensible rather than reckless.

Outline

Today covers a two-toolboxes-on-one-shelf analogy for holding both techniques in mind simultaneously; a practical decision procedure for choosing between them; fractional knapsack, worked completely, showing the surprising result that removing one constraint from Week 9's 0/1 knapsack flips the correct technique from dynamic programming to greedy; why this single constraint matters so much; a consolidated comparison of every proof technique covered across Weeks 9 and 10; "try greedy first" as a defensible practical strategy, together with the caveat that prevents it from being reckless; the MTech-track preview of problems where neither technique alone suffices; an in-class quiz; and a summary.

The two-toolboxes-on-one-shelf analogy

Week 9 built one complete toolbox: dynamic programming, applicable to any problem exhibiting overlapping subproblems and optimal substructure. Week 10 built a second, narrower toolbox: greedy algorithms, applicable only to the smaller set of problems that additionally satisfy the greedy-choice

property, proven via a problem-specific exchange argument. Facing a genuinely new optimization problem, both toolboxes sit on the same shelf, available to reach for — this lecture is about developing the judgment to know which one actually fits, and to recognize the warning signs either way.

The practical decision procedure

The practical decision procedure has five steps. First, confirm the problem has optimal substructure at all, using Week 9's framework — if it does not, neither dynamic programming nor greedy reasoning applies, and a different technique entirely is needed. Second, attempt to identify a locally obvious greedy choice, exactly as this unit's three examples did. Third, actively attempt to construct a counterexample defeating that greedy choice, using the same method Week 9 Lecture 4 and Week 10 Lecture 1 both demonstrated — this step is not optional, and skipping it is the single most common source of a subtly incorrect greedy algorithm shipped into production. Fourth, if a counterexample is found, the greedy-choice property genuinely fails, and dynamic programming is the correct fallback. Fifth, if no counterexample is found after genuine, serious effort, attempt a formal exchange-argument proof before trusting the greedy strategy in any consequential setting.

Fractional knapsack: removing one constraint

Week 9 Lecture 2's knapsack problem required each item to be a binary, 0/1 choice — fully included or fully excluded, with no partial selections permitted. Fractional knapsack changes exactly one rule: items can now be broken into any fraction, taking, for instance, 30% of an item's weight and correspondingly receiving exactly 30% of its value. This single rule change — allowing fractional selection — is the entire difference between the two problem statements, and yet it changes the correct solution technique entirely, as the next slides demonstrate.

Fractional knapsack's greedy strategy

```
def fractional_knapsack(weights, values, W):
    items = sorted(range(len(weights)), key=lambda i: values[i] / weights[i], reverse=True)
    total_value = 0
    remaining = W
    for i in items:
        if weights[i] <= remaining:
            total_value += values[i]                # take the WHOLE item
            remaining -= weights[i]
        else:
            fraction = remaining / weights[i]
            total_value += values[i] * fraction        # take a FRACTION of it
```

```
            remaining = 0
            break
    return total_value
```

The greedy strategy for fractional knapsack sorts items by value-per- weight ratio, descending, and greedily takes as much of the best-ratio item as fits within the remaining capacity, then proceeds to the next best ratio, potentially taking only a fraction of an item to exactly fill the remaining capacity. Unlike Week 9's plausible-but-wrong strategies for the 0/1 variant, this specific greedy strategy is genuinely, provably optimal for the fractional version — the exchange argument (an item's fraction can always be adjusted continuously, unlike a 0/1 all-or-nothing choice) is sketched in the next slide.

Why fractional's greedy is correct, but 0/1's greedy is not

The reason fractional knapsack's greedy strategy is correct while 0/1 knapsack's analogous strategies are not comes down to exactly one structural difference. In the fractional case, if a better-ratio item is not fully used while some worse-ratio item is included, shifting a small fraction of weight from the worse item to the better one strictly improves total value — a continuous exchange is always available, exactly the kind of argument Lecture 1 and Lecture 2 relied on. In the 0/1 case, items cannot be fractionally swapped at all — an item is either fully included or fully excluded, so this continuous exchange argument simply cannot be constructed. The 0/1 constraint removes exactly the flexibility the exchange argument depends on, which is precisely why Week 9's dynamic programming approach remains necessary for that variant, while this lecture's greedy approach suffices here.

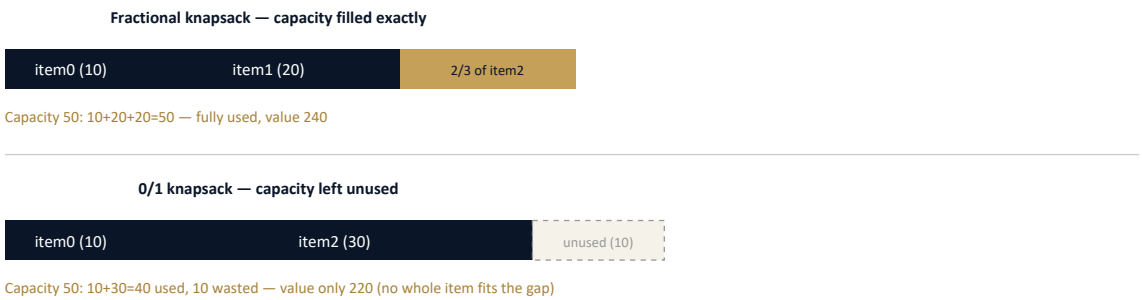
Verifying with a concrete numeric example

```
Items: weight=[10,20,30], value=[60,100,120]. Capacity=50.
Ratios: 60/10=6, 100/20=5, 120/30=4  → sorted: item0(6), item1(5), item2(4)

Fractional: take all of item0 (w=10,v=60, remaining=40)
            take all of item1 (w=20,v=100, remaining=20)
            take 20/30 of item2: v=120*(20/30)=80
Total value: 60+100+80=240

0/1 (Week 9's DP would be needed to confirm, but by inspection):
  items 0+1: weight=30, value=160, leaves capacity 20 unused
  items 0+2: weight=40, value=180
  Best 0/1 combination found: 220 (items 0 and 2), STRICTLY LESS than fractional's 240
```

Verifying both variants on the same numeric instance: fractional knapsack achieves total value 240, using all of items 0 and 1 plus exactly two-thirds of item 2. The 0/1 variant, unable to take a partial

item 2, achieves at best 220, using items 0 and 2 whole and leaving item 1 out entirely. This confirms concretely, not merely abstractly, that allowing fractional selection can achieve a strictly better result than the 0/1 constraint permits — the two problems are genuinely different, not merely differently phrased versions of the identical problem.

The picture: same items, two different optimal strategies



Seeing both solutions side by side on the identical instance makes the consequence of the single rule change visually immediate: fractional knapsack fills the capacity exactly, using a fraction of its last item, while 0/1 knapsack is forced to leave capacity unused, since no whole item exactly fits the remainder.

Tracing the value-per-weight sort explicitly

```
Items: weight=[10,20,30], value=[60,100,120]
Ratios: item0 = 60/10 = 6.0
        item1 = 100/20 = 5.0
        item2 = 120/30 = 4.0
Sorted descending by ratio: item0 (6.0), item1 (5.0), item2 (4.0)
```

Computing the value-per-weight ratio for each item explicitly: item 0 offers 6 units of value per unit weight, item 1 offers 5, and item 2 offers only 4. Sorting by this ratio, descending, is the entire greedy rule — always take as much as possible of whichever item currently offers the best “value density,” which is exactly the metric the continuous exchange argument from two slides ago relies on to guarantee optimality.

Consolidating every proof technique from Weeks 9-10

This table consolidates every problem covered across both weeks against the technique that actually solves it correctly, and why. Dynamic programming problems share overlapping subproblems and either lack the greedy-choice property (proven, as in coin change) or have not had it verified. Every

greedy-solved problem has its own genuinely distinct exchange argument, tailored to that problem's specific structure — swapping one activity, swapping sibling tree positions, the cut property, or a continuous fractional exchange. No single argument transfers between rows; each was independently constructed and verified.

"Try greedy first" as a practical strategy, with its caveat

In practice, attempting a greedy strategy first is often a reasonable starting point — greedy algorithms are typically simpler to implement and considerably faster than dynamic programming, when they happen to be correct. The non-negotiable caveat: never ship a greedy algorithm into a consequential production system without either a genuine correctness proof or exhaustive, deliberate counterexample-searching that failed to find one. "It passed my test cases" is not equivalent to either — the coin-change counterexample from Week 9 and this unit's shortest-duration-first counterexample were each found on specific, sometimes quite small inputs that an inadequate test suite could easily have missed entirely.

MTech addition — problems needing neither alone

Some genuinely harder problems require combining both techniques rather than choosing one exclusively — certain scheduling variants and some graph problems use a greedy pre-processing step to reduce the problem's effective size or eliminate options that are provably dominated by better alternatives, and then apply dynamic programming to solve the remaining, genuinely harder core exhaustively. Recognizing when this hybrid pattern is appropriate is an advanced skill this course does not develop fully, but it is worth knowing such combinations exist, rather than assuming every optimization problem must be purely one technique or purely the other.

Cost comparison: fractional greedy vs. 0/1 dynamic programming

Beyond the surprising result that these near-identical problems need different techniques, it is worth noting the cost difference too: fractional knapsack's greedy solution costs only $O(n \log n)$, for the initial sort, while 0/1 knapsack's dynamic programming solution costs $O(nW)$, the pseudo-polynomial cost identified in Week 9 Lecture 2. The 0/1 constraint does not merely change which technique is correct — it also makes the resulting correct solution genuinely more expensive to compute, a second, compounding consequence of removing the ability to take fractional amounts.

Exercise

As an exercise, apply the five-step decision procedure in writing to a new scheduling problem: given jobs each with a deadline and a penalty for missing it, schedule jobs into single time slots to minimize total penalty. Determine whether a greedy strategy works, either by constructing a counterexample or

by attempting a genuine exchange- argument proof. Separately, implement `fractional_knapsack` and confirm it achieves total value 240 on this lecture's worked numeric example.

Where this judgment applies beyond this course

This exact judgment — greedy or dynamic programming — is a common and deliberate test in technical interviews, precisely because the correct answer is often not obvious from a problem's surface phrasing alone, exactly as this lecture's fractional-versus-0/1 knapsack comparison demonstrated. Real production incidents have genuinely resulted from shipped greedy heuristics that performed correctly across an entire testing suite but failed on a specific, adversarial, or simply unanticipated input in production — the coin-change and shortest- duration-first counterexamples in this unit were not contrived purely for pedagogical effect; they represent a real and recurring category of software defect. This unit's actual deliverable is not "know these three specific greedy algorithms" — it is the transferable judgment to determine, for any new optimization problem encountered in the future, which technique is actually, provably correct.

Quiz — greedy or DP?

Given the problem "given a set of intervals, find the minimum number of points such that every interval contains at least one point," apply this lecture's decision procedure and determine whether greedy or dynamic programming is the appropriate technique, before checking the answer.

Quiz — answer

This problem is structurally almost identical to Lecture 1's activity selection: sorting intervals by their endpoint and greedily placing a point at the endpoint of the earliest-ending interval not yet covered, then skipping every interval already containing that point, mirrors Lecture 1's strategy directly. The correct answer is greedy, and an exchange argument nearly identical to Lecture 1's applies: the earliest-ending interval must be covered by a point placed at or before its own end, and placing it exactly at that endpoint never harms the ability to cover later intervals. Recognizing structural similarity to a previously proven problem is a legitimate and useful shortcut for generating a hypothesis — but the proof must still be explicitly verified for the new problem, never simply assumed to transfer by resemblance alone.

A closing look back across the entire course so far

It is worth pausing to see the shape of the entire course so far. Weeks 1 through 4 established how to build and store data at all, with rigorously proven costs — Big-O as the measuring tool, then arrays, stacks and queues, and linked lists as the storage options. Weeks 5 and 6 established how to put data

into order, at a range of guaranteed costs, from the simple sorts' $O(n^2)$ up to merge sort and quicksort's $O(n \log n)$. Weeks 7 and 8 established how to organize data for fast, specialized access — trees for maintained ordering, hash tables and heaps for narrower but faster guarantees. Weeks 9 and 10, just closing, established how to make genuinely optimal decisions over data — either exhaustively via dynamic programming, or provably safely via a locally greedy choice, when the stronger greedy-choice property can actually be established. Next week's graphs are the setting where many of these ideas — traversal, trees, greedy optimization — recombine.

Summary

The practical decision procedure for a new optimization problem: confirm optimal substructure holds at all, identify a candidate greedy choice, actively and seriously search for a counterexample defeating it, and either construct a genuine proof or fall back to dynamic programming. Fractional knapsack, correctly solved by greedy reasoning, and 0/1 knapsack, requiring Week 9's dynamic programming, differ by exactly one constraint — a concrete, memorable lesson in how consequential a single rule change can be for which technique actually applies. Every greedy algorithm covered across this unit required its own distinct, problem-specific exchange argument, and none of them transferred correctness by analogy alone. "Try greedy first" remains a defensible practical strategy, but only when paired with genuine, deliberate counterexample-searching or a real proof — never with test-suite-passing alone as the standard of correctness. This closes Weeks 9 and 10. Next week introduces graphs formally, the structure underlying this unit's minimum spanning tree algorithm, and the setting for the remaining algorithmic techniques this course covers.